

# *Threshold and policy-based cryptography*

*(Secret sharing applications and extensions)*

# From theory to practice

## → Practical applications of secret sharing “machinery”

## → Several Scenarios

- ⇒ group-oriented crypto
  - Threshold Encryption
  - Threshold Signatures
  - Fancy constructions
- ⇒ Attribute-based cryptosystems
  - (monotone) access control policies
  - More expressive than “just” thresholds
- ⇒ Delegation and distribution of authority
  - Avoid points of failure, improve resilience
  - Capture resistance
  - Delegate security functionalities to proxies

# **Threshold encryption**

**(use-case: ElGamal)**

# Public Key Encryption with DLOG

## → Recap:

- ⇒ 1976: Diffie-Hellman used DLOG but «only» for key agreement
- ⇒ 1977: RSA solved public key encryption, but with different assumption (factorization)

## → Question: public key encryption with DLOG → how to?

## → Answer: El Gamal, 1985

- ⇒ turn DH key agreement into an encryption protocol
- ⇒ Construction: a Columbus egg! 😊

# Why DLOG cryptosystems?

Conveniently implemented over Elliptic Curve Groups

And why we want EC cryptography, then?

| <u>Symm key equiv</u> | <u>modulus size</u>      | <u>Elliptic Curve</u> |
|-----------------------|--------------------------|-----------------------|
| 80 bits               | 1024 bits                | 163 bits              |
| 128 bits              | 3072 bits                | 283 bits              |
| 256 bits              | <b><u>15360</u></b> bits | 571 bits              |

**EC scales better than ordinary modular groups  
with increased security parameter!!**

**Obvious long term deployment choice!**

# ElGamal - background

## → **Asymmetric cryptosystem**

- ⇒ Public key system, 1985, Taher Elgamal
- ⇒ Based on Discrete log
  - Different “hard” problem than RSA (factoring)
- ⇒ Dlog based Public Key (asymmetric) cryptosystem

## → **Inspired by DH**

- ⇒ DH = key agreement, NOT a cipher
- ⇒ ElGamal: turns DH idea into an asymmetric cipher

## → **Probabilistic cipher**

- ⇒ Multiple encrypt of same message = different ciphertexts
- ⇒ Semantic security (under DDH assumption)
- ⇒ Price to pay:  $1 \rightarrow 2$  ciphertext expansion

# ElGamal - sketch

$p$  large prime

$g$  Group generator

→ **All operations mod  $p$**

$s$  Private key

$h = g^s$  Public key

$r$  random

Creative way to “reuse” Diffie-Hellman!

$g^s, g^r$  : known to all ( $g^s$  public,  $g^r$  in ciphertext)

$s$  : known only by RX (secret key)

$r$  : known only by TX (random)

$g^{sr}$  : No way for anybody else to compute

Encrypt  $(R, c) = (g^r, m \cdot h^r)$

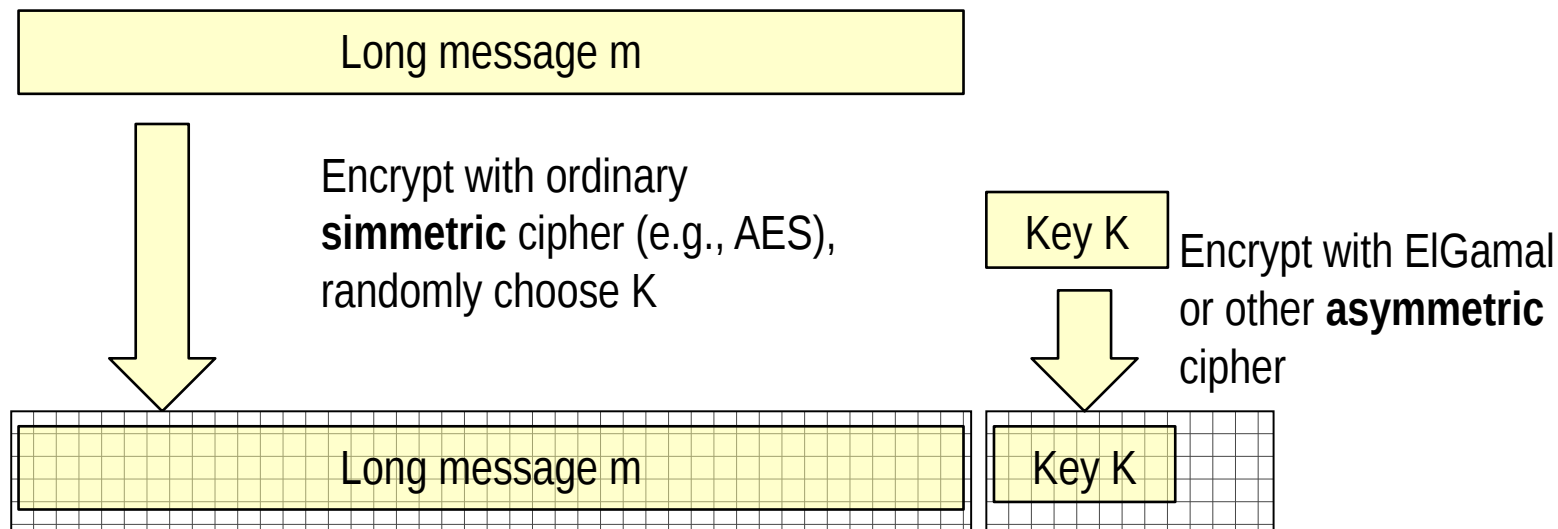
Decrypt  $m = c \cdot R^{-s} = \frac{c}{(g^r)^s} = \frac{m \cdot h^r}{g^{rs}} = \frac{m \cdot g^{sr}}{g^{rs}}$

# Anymmetric ciphers: “hybrid” usage

## → Message must be point in group

- ⇒ E.g., a number between 1 and  $p-1$  if using mod  $p$  integers
- ⇒ Prime  $p$ : for instance 1024 bits
- ⇒ But messages can be much longer than that!

## → Hybrid encryption





# How El-Gamal-type crypto is used today?

## ECIES = Hybrid Encryption in 5G

### → SUPI (IMSI) → SUCI

- ⇒ Subscriber Concealed Identifier
- ⇒ Public key encryption of SUPI (IMSI)!
- ⇒ Use home network public key pre-loaded in SIM

### → Approach: ECIES

- ⇒ Elliptic Curve Integrated Encryption Scheme

### → Idea (without EC details):

Ephemeral  $g^x$



$K = \text{HKDF}(g^{(HN \cdot x)}) \longrightarrow \text{AES}_K(\text{SUPI}) \longrightarrow \text{HMAC}$

# Threshold ElGamal

## → **Decryption when threshold # of receivers cooperate**

- ⇒ Message can be read only by  $t$  cooperating parties
- ⇒ Nobody has private key (!) - safer

## → **First idea:**

- ⇒ Distribute shares of private key  $s$ 
  - E.g. using Pedersen for  $s$  initially unknown
- ⇒ Reconstruct  $s$  when decryption needed

## → **But...**

- ⇒ Works only once!
  - Once secret key reconstructed, ANYTHING can be decrypted!

## → **Can we decrypt one message, and guarantee confidentiality for any other message?**

# Towards the solution...

Encrypt  $(g^{r_1}, m_1 \cdot h^{r_1})$

Decrypt  $m_1 = \frac{c}{(g^{r_1})^s}$

Decrypt  $m_2 = \frac{c_2}{(g^{r_2})^s}$

- How to decrypt  $m_1$  and NOT  $m_2$ ?
- How to compute denominator for  $r_1$  s.t. computation for a different  $r_2$  is not revealed?

General property of exponentiation :  $A^x = A^{x_1} \cdot A^{x_2} = A^{x_1 + x_2}$

- Can we interpolate “shares” at exponent?

# Getting closer...

## → Usual polynomial

$$p(x) = s + a_1x + a_2x^2 + \dots + a_{t-2}x^{t-2} + a_{t-1}x^{t-1}$$

## → Usual shares

$$(x_i, y_i) \quad y_i = p(x_i)$$

## → Usual reconstruction formula...

$$s = \sum_{\text{shares } x_i} y_i \Lambda_{x_i} \quad \text{with} \quad \Lambda_{x_i} = \Lambda_{x_i}(0) = \prod_{\text{shares } x_k \neq x_i} \frac{x_k - x_i}{x_i - x_k}$$

## → What if done at exponent?

$$\prod A^{y_i \Lambda_{x_i}} = A^{\sum y_i \Lambda_{x_i}} = A^s$$

# Actual solution

- Each party owns one share  $(x_i, y_i) \quad y_i = p(x_i)$
- Party gets  $g^r$  from ciphertext  $(g^r, m \cdot h^r)$
- Computes as usual Lagrange coefficients  
 $\Rightarrow$  Depend only on (known) x-axis share values  $\Lambda_{x_i} = \prod_{\text{shares } x_k \neq x_i} \frac{x_k - x_i}{x_i - x_k}$
- Computes exponent share  $(g^r)^{y_i \Lambda_{x_i}}$
- Sufficient # of shares permit to reconstruct decryption term  

$$\prod (g^r)^{y_i \Lambda_{x_i}} = (g^r)^{\sum y_i \Lambda_{x_i}} = (g^r)^s = g^{rs}$$
- Finally factor away decryption term  $\frac{m \cdot h^r}{g^{rs}} = \frac{m \cdot g^{sr}}{g^{rs}} = m$

# **Threshold signature**

**(use-case: RSA)**

# Why a threshold signature?

- **Any  $t$  out of  $n$  members of a group can sign a message**
  - Validity of (signed) message endorsed by multiple “notaries”
  - Group member certified by other  $t$  members
  - Place trust in more than ONE Certification Authority
  - Etc
- **If less than  $t$  members, impossible to forge a signature**
- **Further requirements**
  - ⇒ Should reuse existing signature approaches
    - (not a new signature type)
  - ⇒ Size should not “blow up” with  $t$

# **RSA signature**

**→ Large primes  $p, q$ ;  $N=pq$**

$$\Rightarrow \phi(N) = (p-1)(q-1)$$

**→ Pick  $e$  s.t. coprime to  $\phi(N)$**

**→ Compute  $d = e^{-1} \bmod \phi(N)$**

$$\Rightarrow \text{i.e. } e \cdot d = 1 \bmod \phi(N)$$

**→ Signing a message:**

$$\Rightarrow [m, H(m)^d]$$

**→ Verifying signature:**

$$\Rightarrow H(m) = (H(m)^d)^e$$



# Threshold RSA

## (multi-signatures)

→ Usual approach (?)

→ Dealer:  $f(x) = d + a_1x + a_2x^2 + \dots + a_{t-2}x^{t-2} + a_{t-1}x^{t-1}$

→ Share of  $P_i$   $(x_i, y_i = f(x_i)) \bmod \phi(N)$

→ Message to be signed  $m, H(m)$

→ Signature share  $H(m)^{y_i \Lambda_{x_i}} \bmod N$

→ Valid signature  $\prod H(m)^{y_i \Lambda_{x_i}} = H(m)^{\sum y_i \Lambda_{x_i}} = H(m)^d$

What's wrong???

What differs with RSA wrt previous cases?

# Computing signature share...

(Devil is in the details)

→ Party i must compute  $H(m)^{y_i \Lambda_{x_i}} \bmod N$

→ Where  $\Lambda_{x_i} = \prod_{\text{shares } x_k \neq x_i} \frac{x_i - x_k}{x_i - x_k} = \frac{\alpha_{x_i}}{\beta_{x_i}}$

→ Hence  $H(m)^{y_i \Lambda_{x_i}} = H(m)^{y_i \alpha_{x_i} \beta_{x_i}^{-1}} \bmod N$

→ where  $\beta_{x_i} \cdot \beta_{x_i}^{-1} = 1 \bmod \phi(N)$

- 1) We CANNOT compute inverse without  $\phi(N)$  → factorization needed → RSA breaks!
- 2) If  $\beta$  happens to be even (as it may well be), inverse does NOT exist

# How to avoid inverses?

(brilliant remark!!)

→ Assume usual  $x_i=i$ , and  $L$  players

→ Look at Lagrange polynomial denominator

⇒ Worst case of interpolation on all  $L$  shares:

$$\Lambda_i(x) = \prod_{\text{shares } k \neq i} \frac{x - k}{i - k} = \frac{\text{something}(x)}{(i-1)(i-2)\dots(i-(i-1))(i-(i+1))\dots(i-L)}$$

→ But denominator surely divides  $i!(L-i)!$

→ Which in turns surely divides  $L!$  ☺

→ Consequence:

$$L! \cdot \Lambda_i(x) = \text{surely integer} = \overline{\Lambda}_i(x)$$

# So what?

→ **Dealer:**  $f(x) = d + a_1x + a_2x^2 + \dots + a_{t-2}x^{t-2} + a_{t-1}x^{t-1}$

→ **Share i:**  $(i, y_i = f(i)) \bmod \phi(N)$

→ **Compute signature share as:**  $H(m)^{y_i \bar{\Lambda}_{x_i}} \bmod N$

- ⇒ no inverse needed anymore
- ⇒ Lagrange are NOW integers

→ **Construct “signature” (?) as**

$$\prod H(m)^{y_i \bar{\Lambda}_{x_i}} = H(m)^{L! \sum y_i \bar{\Lambda}_{x_i}} = H(m)^{d \cdot L!} \bmod N$$

**Not yet our signature: extra factor L! at exponent...**

**How to get rid of this?? (cannot do mod  $\phi(N)$  inverses: back to the start!)**

# Not really...

## → Remember RSA common modulus attack

- Alice encrypt message  $m$  with public key  $e_a$
- Bob encrypt SAME message  $m$  with different public key  $e_b$
- Module  $N$  is the same
- $\gcd(e_a, e_b) = 1$

⇒ Then

- Message can be easily decrypted using Extended euclidean algorithm:
  - » Find  $r, s$  s.t.  $e_a \cdot r + e_b \cdot s = \gcd(e_a, e_b) = 1$

# **RSA common modulus attack...**

**→ You know  $e_a, e_b$  (co-primes)**

**→ Determine  $r, s$  from Extended Euclidean Algorithm s.t.**

$$e_a \cdot r + e_b \cdot s = 1$$

**→ You are given encrypted messages  $m^{e_a}; m^{e_b}$**

**→ You wish to know  $m$ , but no decrypt key...**

**→ Not needed! Decrypt message as**

$$\left(m^{e_a}\right)^r \cdot \left(m^{e_b}\right)^s = m^{e_a \cdot r + e_b \cdot s} = m$$

# ...used for a good purpose here!

let  $L! = \Delta$

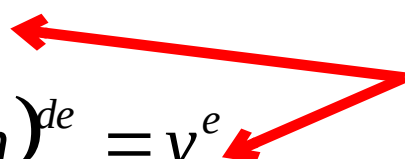
we have  $H(m)^{d \cdot L!} = H(m)^{d \cdot \Delta}$

we want  $H(m)^d = y$

but we have  $H(m)^{d \cdot \Delta} = y^\Delta$

we also have  $H(m) = H(m)^{de} = y^e$

We have cast  
our problem as  
an RSA common  
modulus attack!



if  $e$  and  $\Delta$  are coprime, we can apply "attack"

$\Rightarrow$  and hence derive our signature  $H(m)^d = y$

**If public key  $e$  is a prime larger than  $L$ , then this surely works!**

**Besides this, RSA signature remains perfectly standard!**

# Caveats

## → **Previous presentation largely simplified**

- ⇒ Actual proposal by Victor Shoup, 2000, Practical threshold signatures
  - Runs computations on special subgroup (quadratic residues) and relies on strong RSA (safe primes)
  - explicitly includes verifiability and security proofs
- ⇒ However, essence captured by our simplified explanation

## → **Further extended as fully distributed**

- ⇒ Foque+Stern, 2001
- ⇒ Complexity overcome: hard to distributely generate strong RSA module
  - Bypassed in Foque/Stern work by alternative approach